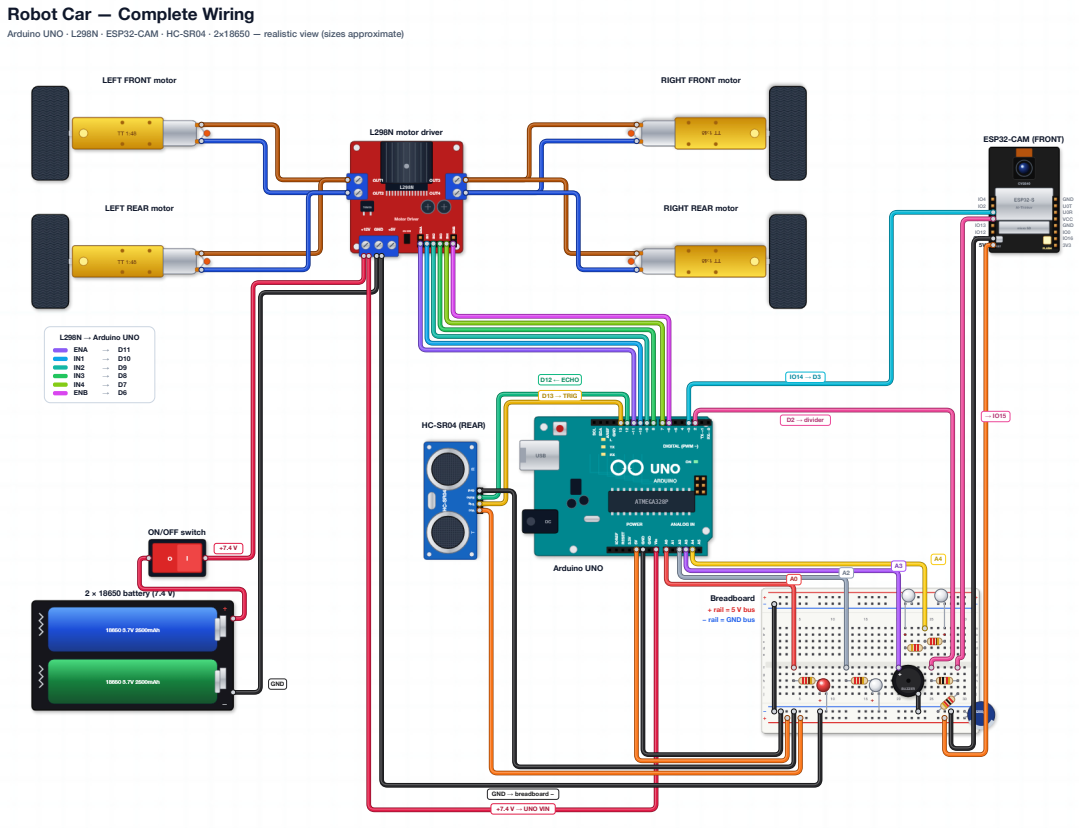
ROBOT CAR PROJECT

STUDENT HANDBOOK

Phone-Controlled Robot Car

Build a 4WD car with live video, a rear parking radar and auto-brake: parts, wiring, assembly, code and tests.



Name _____ Class / Group _____ Date _____

4WD · Arduino UNO · ESP32-CAM
Edition 2026

STUDENT HANDBOOK

Contents

01 **Overview & Parts List** p. 3
What we are building, how it works, and exactly which parts to buy.

02 **Wiring Guide** p. 8
Every connection step by step, with realistic pictures, connection tables and checks.

03 **Assembly Guide** p. 20
Where every part goes on the chassis, how to mount it, and the mechanical safety check.

04 **Software & App Guide** p. 24
Install the tools, upload both boards, connect the phone and master every control.

05 **Testing & Troubleshooting** p. 31
Test one part at a time, record the results, and fix the common problems yourself.

06 **Technical Reference** p. 36
Protocol, safety rules, steering and alarm maths, code structure and extension ideas.

07 **Warm-up Lessons** p. 41
Use the free Learn Arduino website to learn every idea the car's code uses, then test it on a real UNO.

How to use this handbook

Work through the modules in order: **01** parts → **03** assembly → **02** wiring → **04** software → **05** tests. Start with Module **07** (warm-up lessons on the Learn Arduino website) if you are new to Arduino. Module **06** is a reference for reports and improvements. Each module ends with a checklist that your teacher can sign.

Also in this folder

Robot_Car_Wiring_Poster.pdf: A3 poster of the complete wiring (realistic + schematic) for the lab wall.

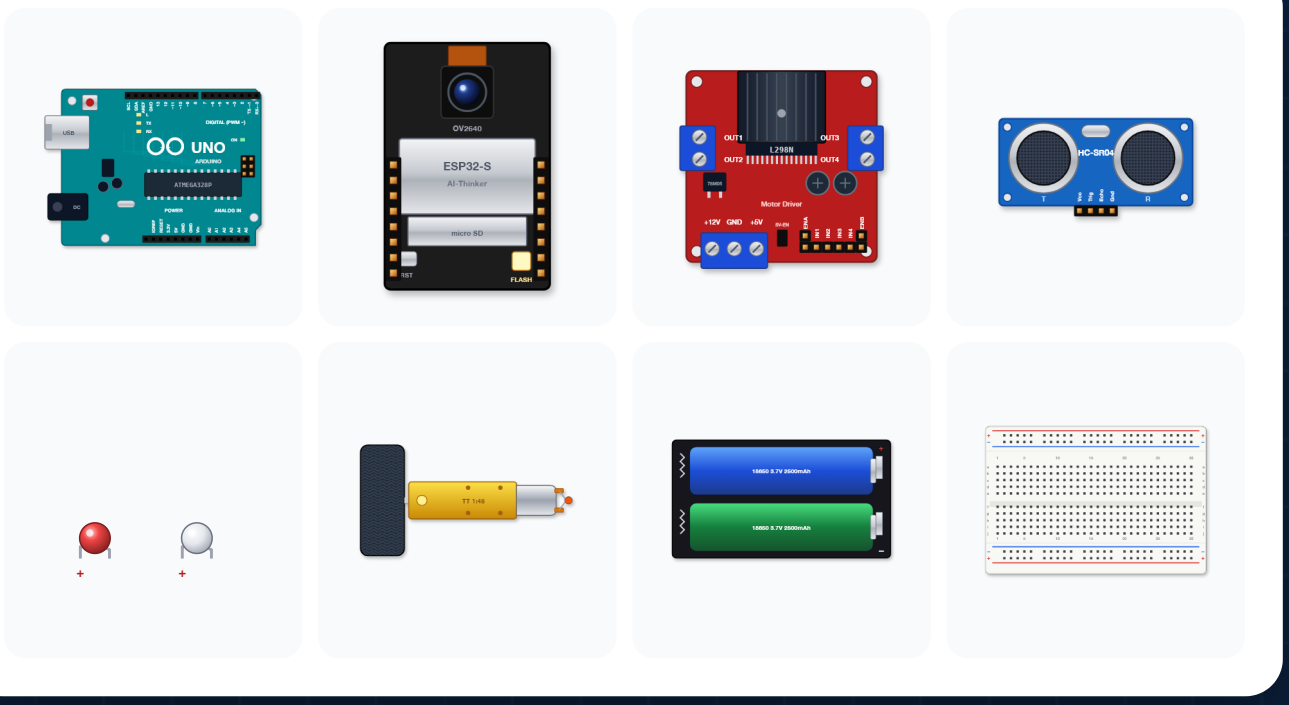
ROBOT CAR PROJECT

MODULE 01

01

Overview & Parts List

What we are building, how it works, and exactly which parts to buy.



Name _____ Class / Group _____ Date _____

4WD · Arduino UNO · ESP32-CAM
Edition 2026

MODULE 01

1 Project overview

In this project you build a **four-wheel-drive robot car that you drive from a smartphone**. A camera at the front sends live video to the phone. A sensor at the back measures the distance to obstacles. When you reverse, the car warns you like a real car's **parking sensor**: a buzzer beeps and a red light flashes, faster and faster as you get closer, and the car brakes by itself before it hits anything. The phone shows the measurement as a **radar**.

LEARNING OBJECTIVES · ABOUT 45 MIN

By the end of this module you can:

- Explain the job of every part in the car and how they talk to each other
- Choose correct parts (voltage, current, type) and avoid common bad buys
- Read a power budget and explain why the car uses two power rails (7.4 V and 5 V)
- Check your kit against the parts list before you start building

What the finished car can do

Phone control
Joystick in the phone browser. No app to install.

Live video
The camera at the front streams to the phone.

Parking alarm
Buzzer and LED beep faster as an obstacle gets closer.

Auto-brake
The car refuses to reverse into an obstacle.

Radar display
Rear distance shown as a radar on the phone.

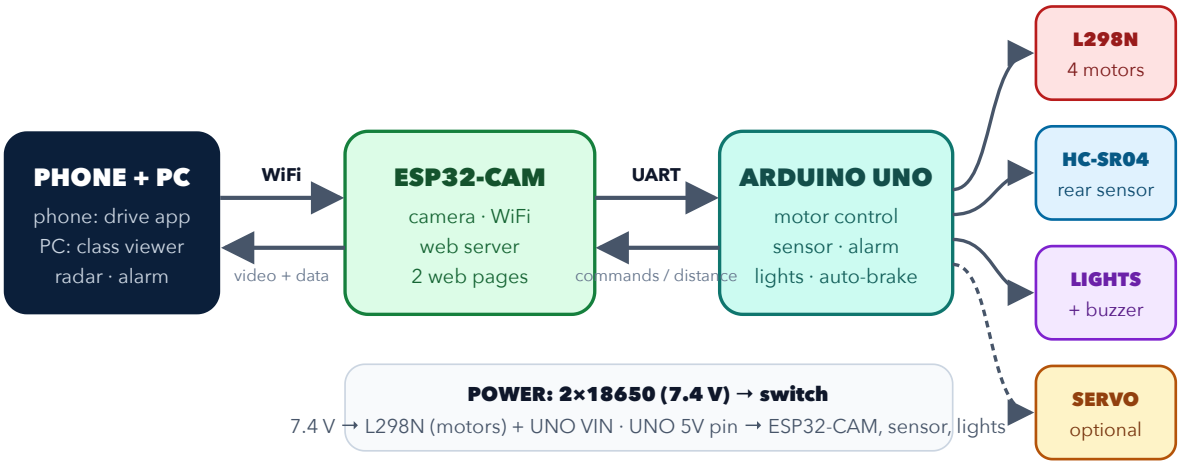
Headlights
Two white front lights: OFF, ON or HAZARD from the app.

Classroom viewer
A second page for a PC or projector: video, radar, chart, status.

Photos & video
Save photos on the phone, record video on the PC.

Fail-safe
Stops by itself if WiFi or the phone is lost.

How it works



Why two small computers?

The **ESP32-CAM** is great at WiFi and video, but it has only about 4 free pins and is busy streaming. The **Arduino UNO** handles the motors, the sensor and the alarm in real time. If WiFi lags, the parking alarm and auto-brake still react instantly.

Two power rails

7.4 V from the batteries goes to the motor driver and to the UNO's **VIN** pin. The UNO's own regulator makes the **5 V** for all the electronics. A big **2200 µF capacitor** at the ESP32-CAM covers the short dips when the motors start, so the camera does not reboot.

MODULE 01

2 Parts list

Everything below is needed for the standard build. Prices are approximate (online shops, 2026) and only for planning.



4WD chassis kit

2 acrylic plates, 4 × TT gear motors (3–6 V, 1:48), 4 wheels 65 mm, spacers, screws

Qty 1 kit — $12-18



Arduino UNO R3 (or Nano)

ATmega328P, USB cable. CH340 clones are fine

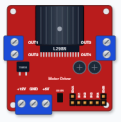
Qty 1 — $5-12



ESP32-CAM (AI-Thinker)

OV2640 camera, 4 MB PSRAM. Get the ESP32-CAM-MB USB board too

Qty 1 + MB — $9-13



L298N motor driver

Dual H-bridge with heatsink, 5 V regulator, ENA/ENB jumpers

Qty 1 — $2-4



HC-SR04 ultrasonic

5 V, 2–400 cm. Mounted at the rear of the car

Qty 1 — $1-2



2 × 18650 + holder

Protected Li-ion 3.7 V, real 2000–3000 mAh, series holder = 7.4 V

Qty 2 + 1 — $8-14



Half breadboard

400 points with + / – rails on both sides

Qty 1 — $1-2



Rocker switch

≥ 3 A, often included in the kit

Qty 1 — $0.3



Active buzzer 5 V

ACTIVE type: it beeps on plain 5 V (test it on a battery)

Qty 1 — $0.5



LEDs 5 mm

1 red (rear warning) + 3 white (reverse light + 2 headlights). Buy a few spares

Qty 4 — $0.4



Resistors

220 Ω × 4 · 1 kΩ × 1 · 2 kΩ × 1 (or 2 × 1 kΩ)

Qty 6 — $0.5

Small parts

Part	Specification	Qty	Used for
Electrolytic capacitor	**2200 µF 16 V** (any value 1000–2200 µF, 10 V or more)	1	Stops the ESP32-CAM rebooting when motors start
Ceramic capacitor	100 nF (marked "104")	4	Soldered across each motor: less electrical noise
Jumper wires	Dupont 20 cm: M-M, M-F, F-F (20 of each)	1 set	Signal connections
Power wire	22 AWG (0.5 mm²) red + black, ~1 m each	–	Battery → switch → L298N, L298N → UNO VIN
ESP32-CAM-MB	Micro-USB programmer board for the ESP32-CAM	1	Uploading code (or use an FTDI adapter)
18650 charger	2-slot Li-ion charger (or 2 × TP4056)	1	Charging the batteries safely

Avoid these bad buys

- **"9900 mAh" 18650 cells**: they are fakes that sag under load and make the ESP32 reboot. Buy known brands (Samsung, LG, Sony/Murata, Molicel).
- **Passive buzzer**: it only clicks on DC. You need an **active** buzzer, which beeps when connected straight to 5 V.
- **ESP32-CAM without PSRAM**: the video then works only at a small size. Look for "4 MB PSRAM".
- **9 V PP3 battery or 4 × AA**: far too weak for 4 motors.

Checklist: check your kit

- [ ] 4 TT motors + 4 wheels + chassis plates
- [ ] Arduino UNO + USB cable
- [ ] ESP32-CAM + camera + MB board
- [ ] L298N motor driver
- [ ] HC-SR04 sensor
- [ ] Active buzzer
- [ ] 2 × 18650 cells (charged) + holder
- [ ] Switch, breadboard, jumper wires
- [ ] LEDs: 1 red + 3 white
- [ ] Resistors 220 Ω ×4, 1 kΩ, 2 kΩ
- [ ] Capacitors 2200 µF + 4 × 100 nF
- [ ] Multimeter + soldering iron

Checked by (student)

Approved by (teacher)

MODULE 01

3 Tools & power budget

Tools

Tool	Needed for	
Multimeter	Checking the battery polarity and the 5 V rail	REQUIRED
Soldering iron + solder	Motor wires and capacitors	REQUIRED
Small screwdrivers	L298N screw terminals	REQUIRED
Wire stripper / cutter	Preparing power wires	REQUIRED
Computer + Arduino IDE 2	Programming both boards	REQUIRED
FTDI USB-serial adapter	Only if you have no ESP32-CAM-MB board	OPTIONAL
Hot glue, zip ties, M3 nylon standoffs	Neat, vibration-proof mounting	OPTIONAL

Power budget: why these parts?

Load	Rail	Typical	Peak
4 × TT motors	7.4 V via L298N	0.6–1.0 A	2.5 A (start / stall)
ESP32-CAM (WiFi + video)	5 V	180 mA	**350 mA spikes**
Arduino UNO itself	5 V	50 mA	60 mA
HC-SR04 + buzzer	5 V	30 mA	45 mA
4 LEDs (rear, reverse, 2 headlights)	5 V	0–60 mA	60 mA
Total from the UNO 5V pin		**≈ 0.3 A**	**≈ 0.5 A** for a moment: OK for the UNO regulator
SG90 servo (optional)	**own 5 V supply**	100 mA	600 mA: too much for the UNO, never on this rail

Why 7.4 V?
The L298N loses about 2 V, so the TT motors receive about **5.4 V**, which is their sweet spot. Two 18650 cells in series give exactly that.

Why the UNO's 5 V, not the L298N's?
The L298N's small regulator gets hot and its 5 V dips when the motors start, so the ESP32-CAM resets ("brownout"). The UNO's regulator is stronger. With the 2200 µF capacitor it keeps the camera running. It gets warm: that is normal.

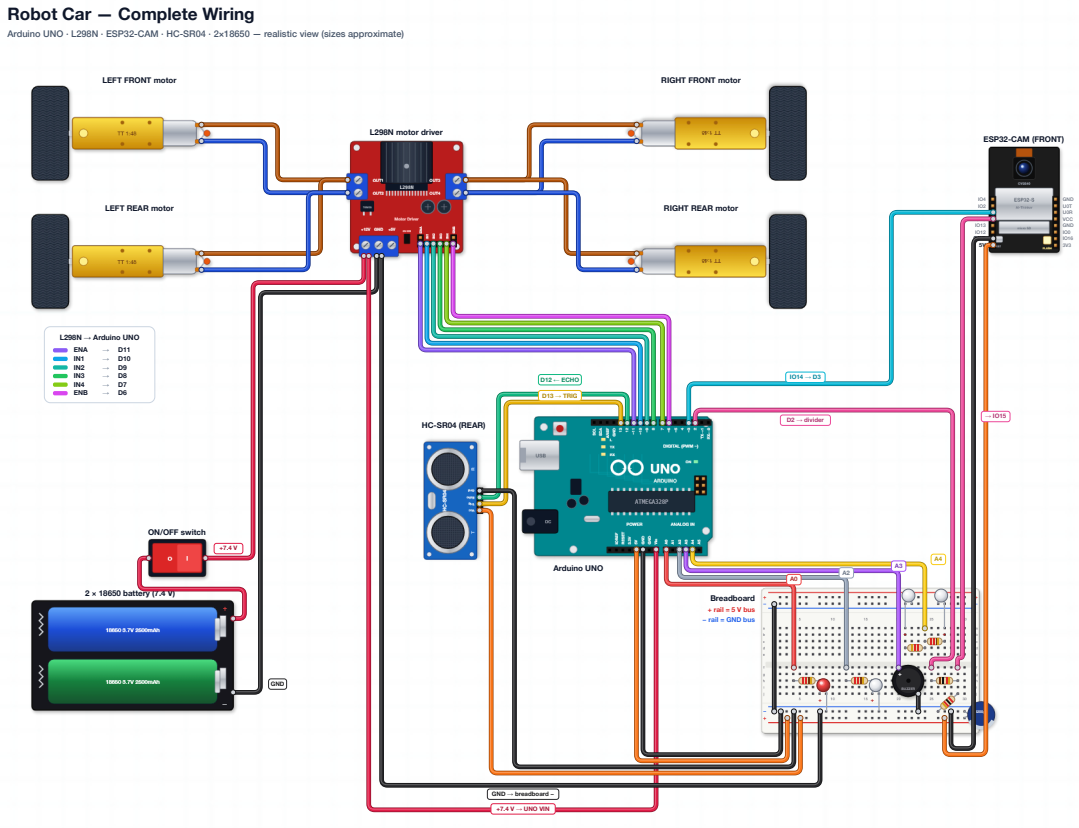
ROBOT CAR PROJECT

MODULE 02

02

Wiring Guide

Every connection step by step, with realistic pictures, connection tables and checks.



Name _____ Class / Group _____ Date _____

4WD · Arduino UNO · ESP32-CAM
Edition 2026

MODULE 02

1 Wiring guide

This module shows **every connection** in the car, one step at a time. Each step has a close-up picture with the wires of that step highlighted, a list of actions and a connection table. Do the steps **in order** and test as you go.

LEARNING OBJECTIVES · ABOUT 120 MIN

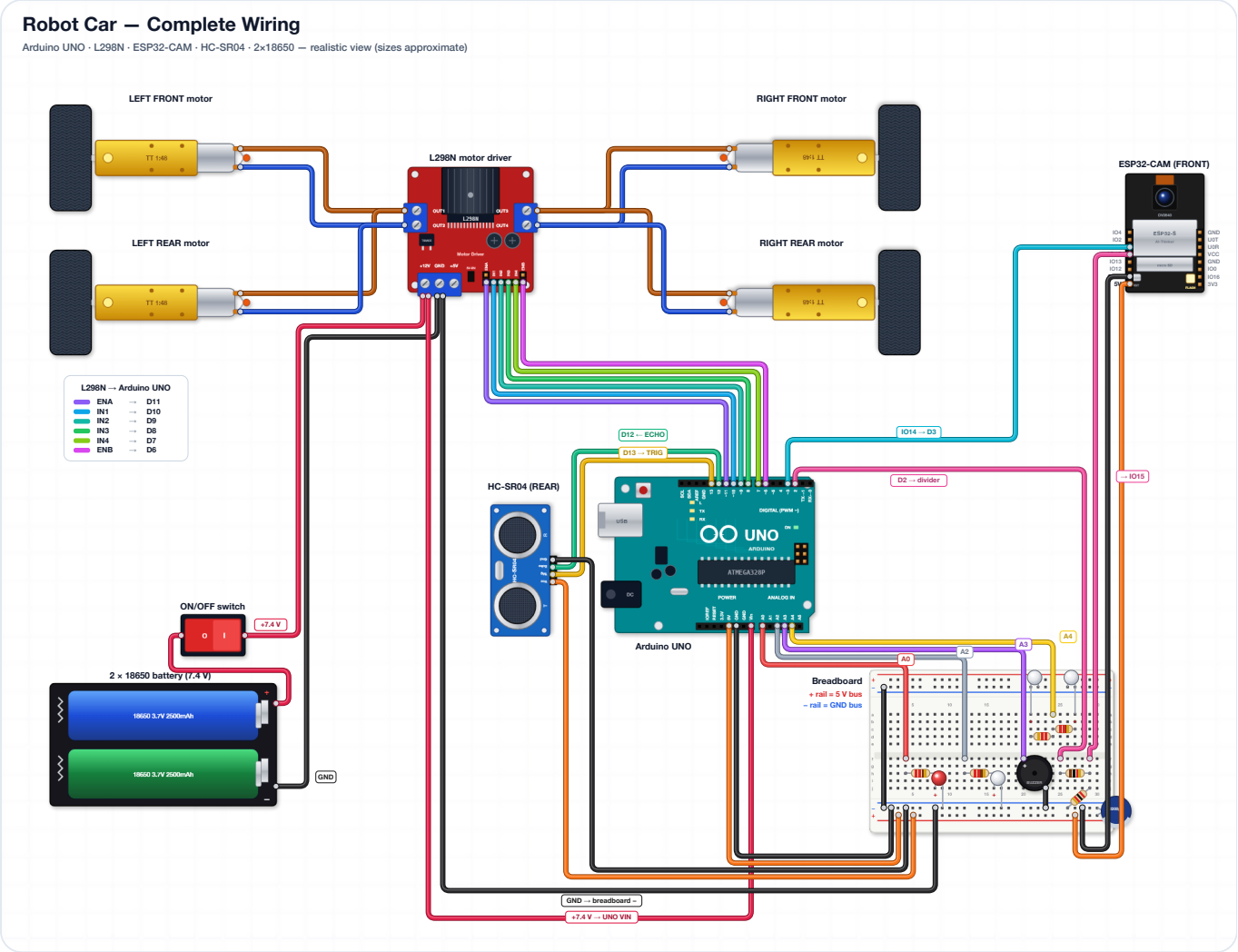
By the end of this module you can:

- Wire a two-rail power system safely (7.4 V and 5 V) with a common ground
- Connect the motors, motor driver, sensor, buzzer and LEDs to the Arduino
- Use a voltage divider to connect a 5 V board to a 3.3 V board
- Check a circuit with a multimeter before powering it

Golden rules: read before you touch a wire

- Always wire with the **battery switched OFF** (better: batteries out of the holder).
- Check the **battery polarity with the multimeter** before the first power-on: battery + must reach the switch, the L298N +12V and the UNO **VIN**.
- **All GNDs are connected together**: battery −, L298N, UNO, ESP32-CAM, sensor, buzzer, LEDs.
- 7.4 V goes **only** to the L298N +12V terminal and the UNO **VIN** pin. Never to a 5V pin or to the ESP32-CAM.
- Never connect a 5 V output straight to an ESP32 pin: use the voltage divider (step 7).

The complete wiring



Complete realistic wiring diagram. An A3 poster version is in **Robot_Car_Wiring_Poster.pdf**.

How to read the pictures

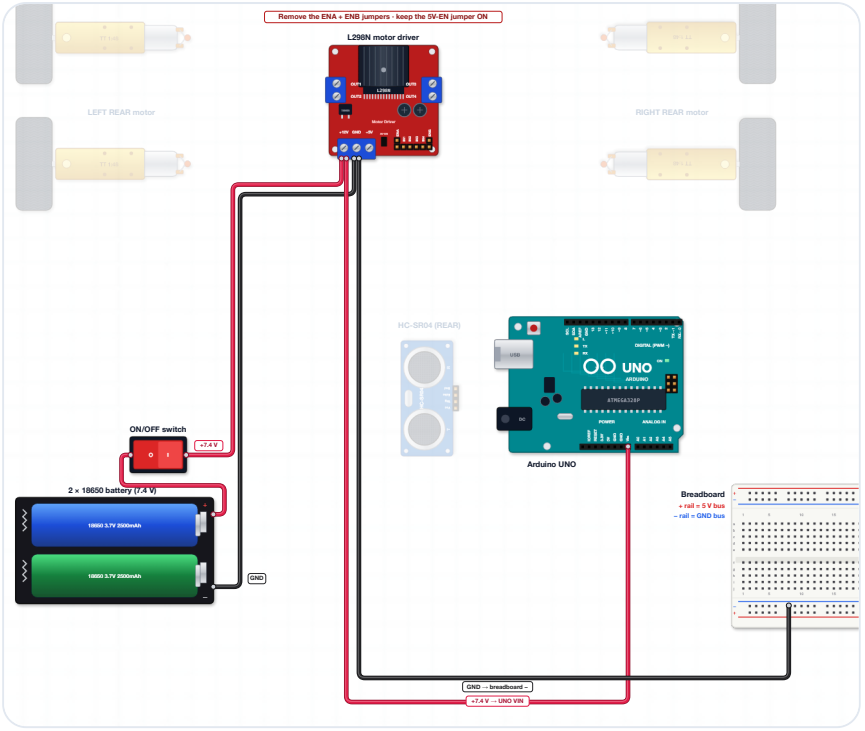
Each coloured line is one jumper wire. Where wires cross they are **not** connected (they are insulated). The breadboard's long **red + rail** is the 5 V bus and the **blue − rail** is the GND bus.

Breadboard rails

On some breadboards the rails are split in the middle. Check with the multimeter (continuity mode) that column 1 and column 30 of each rail are connected. If not, add a short jumper across the gap.

1 Power system

Battery → switch → L298N +12V · second wires: +12V → UNO VIN, GND → breadboard − rail



Two wires in one screw terminal

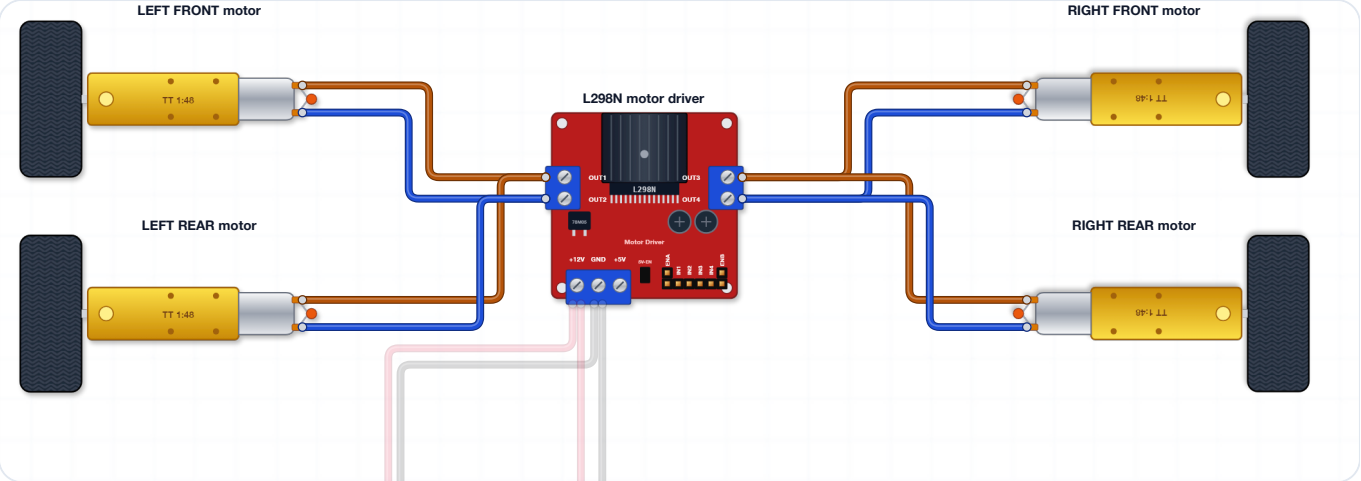
The L298N **+12V** terminal holds two red wires: the one from the switch and the one to the UNO **VIN**. The **GND** terminal holds two black wires: battery − and the wire to the breadboard − rail. Twist the bare ends together, push them in and tighten firmly. No buck converter is needed: the UNO makes the 5 V (step 4).

1. Battery **+** (red) → switch, first pin.
2. Switch second pin → **L298N +12V**.
3. Battery **−** (black) → **L298N GND**.
4. Second red wire: **L298N +12V** → **UNO VIN** (power header, next to GND).
5. Second black wire: **L298N GND** → **breadboard − rail**.
6. On the L298N: **remove the ENA and ENB jumpers** and keep the 5V-EN jumper on. Leave the L298N +5V terminal empty.

	From	To	Note
	Battery +	Switch	22 AWG wire
	Switch	L298N +12V	carries the motor current
	Battery −	L298N GND	
	L298N +12V	`VIN`	7.4 V into the UNO regulator
	L298N GND	Breadboard − rail	= GND bus

2 Motors

Left pair on OUT1/OUT2 · right pair on OUT3/OUT4



The two motors on the same side are wired **in parallel**: both wires of the front motor and both wires of the rear motor go into the same two screws.

1. Solder about 15 cm wires and a **100 nF capacitor** to each motor (see Module 03).
2. Left front + left rear: wire A → **OUT1**, wire B → **OUT2**.
3. Right front + right rear: wire A → **OUT3**, wire B → **OUT4**.
4. Tighten the screws and pull gently on each wire to check it is held.

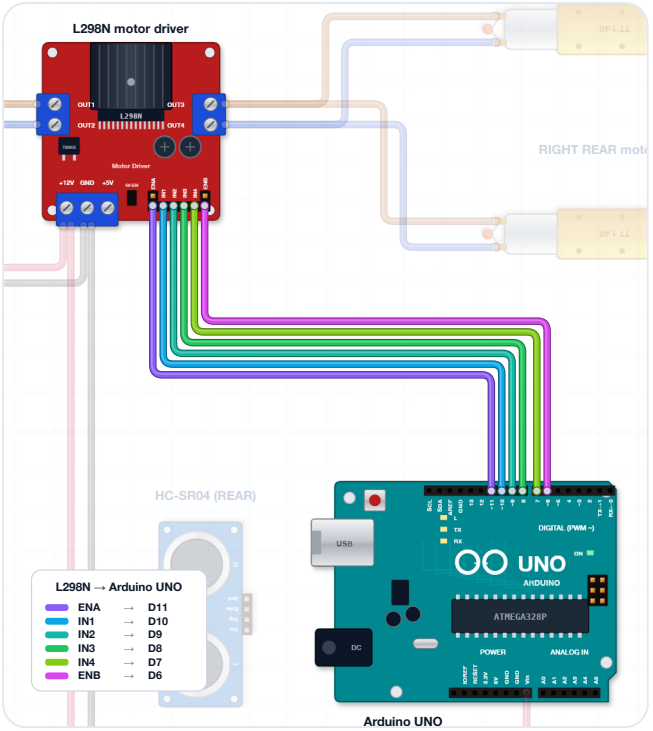
	From	To	Note
	Left motors, wire A	L298N OUT1	both motors
	Left motors, wire B	L298N OUT2	both motors
	Right motors, wire A	L298N OUT3	both motors
	Right motors, wire B	L298N OUT4	both motors

Wrong direction later?

If one motor turns against its partner, swap **only that motor's** two wires. If a whole side is backwards, change `INVERT_LEFT` / `INVERT_RIGHT` in the UNO code.

3 L298N → Arduino UNO

Six control wires, side by side



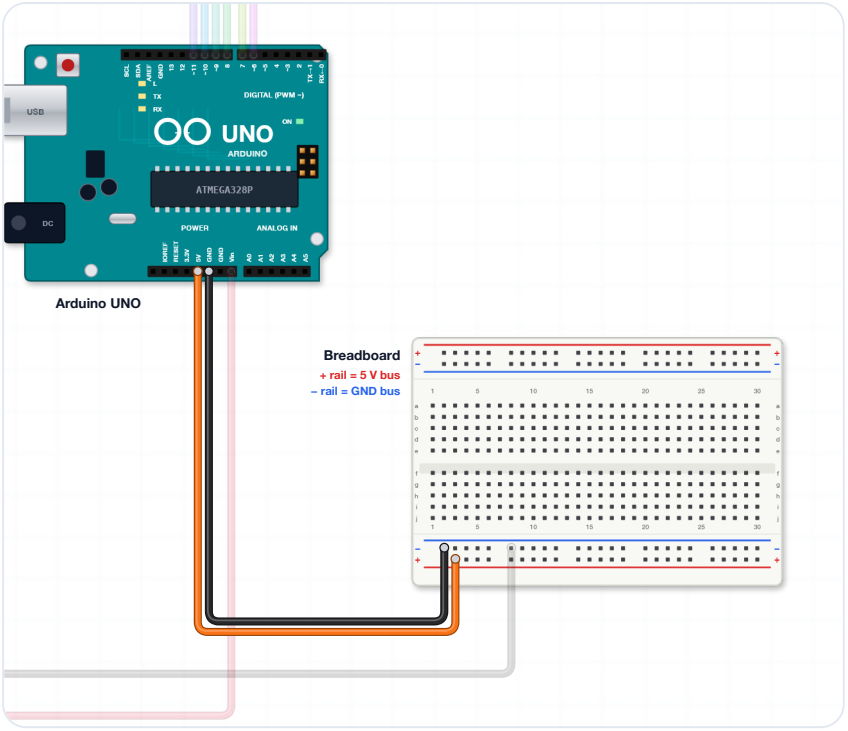
The pins were chosen so the six wires run in the same order on both boards. Use six different colours.

1. ENA → **D11** (PWM ~, left speed).
2. IN1 → **D10**, IN2 → **D9** (left direction).
3. IN3 → **D8**, IN4 → **D7** (right direction).
4. ENB → **D6** (PWM ~, right speed).
5. Test now: upload `tests/01_motor_test` with the wheels in the air (Module 05).

	From	To	Note
	`ENA`	`D11 ~`	left speed (PWM)
	`IN1`	`D10`	left direction
	`IN2`	`D9`	left direction
	`IN3`	`D8`	right direction
	`IN4`	`D7`	right direction
	`ENB`	`D6 ~`	right speed (PWM)

4 5 V rail from the Arduino

UNO 5V and GND to the breadboard rails



The UNO turns the 7.4 V on **VIN** into a steady **5 V**. Its **5V pin** now feeds the breadboard + rail, and that rail powers the ESP32-CAM, the sensor, the buzzer and the LEDs.

1. UNO **GND** → breadboard **– rail**.
2. UNO **5V** → breadboard **+ rail**.
3. Do **not** also plug a power supply into the UNO's DC jack: VIN is already used.
4. Check: battery on, multimeter between the + and – rails = **4.9–5.1 V**.

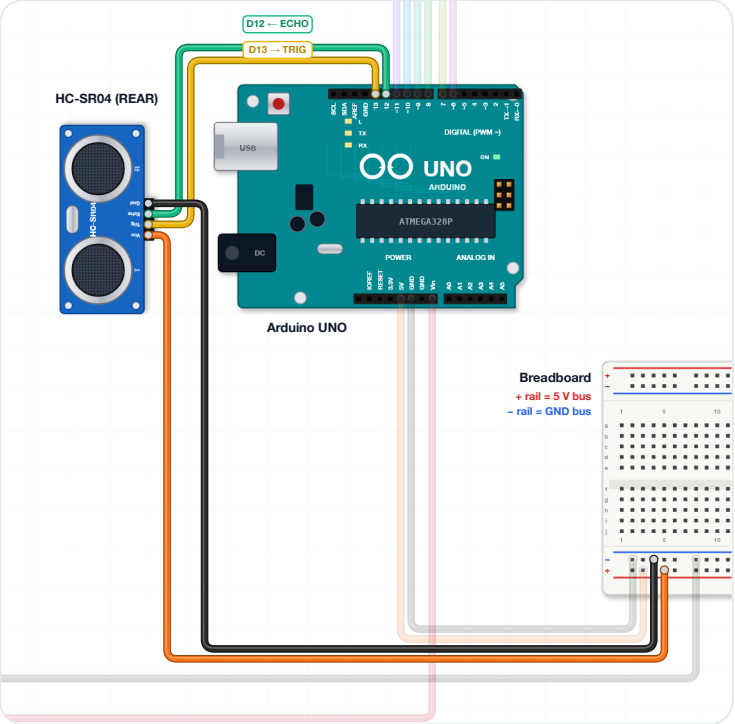
	From	To	Note
▬	`GND`	Breadboard – rail	power header
▬	`5V`	Breadboard + rail	= 5 V bus for everything

Can I keep the USB cable plugged in?

Yes. The UNO picks the stronger supply by itself. On USB only (battery off) the electronics and the camera work but the motors do not move. That is handy while testing.

5 Rear ultrasonic sensor

TRIG → D13 · ECHO → D12 · power from the rails



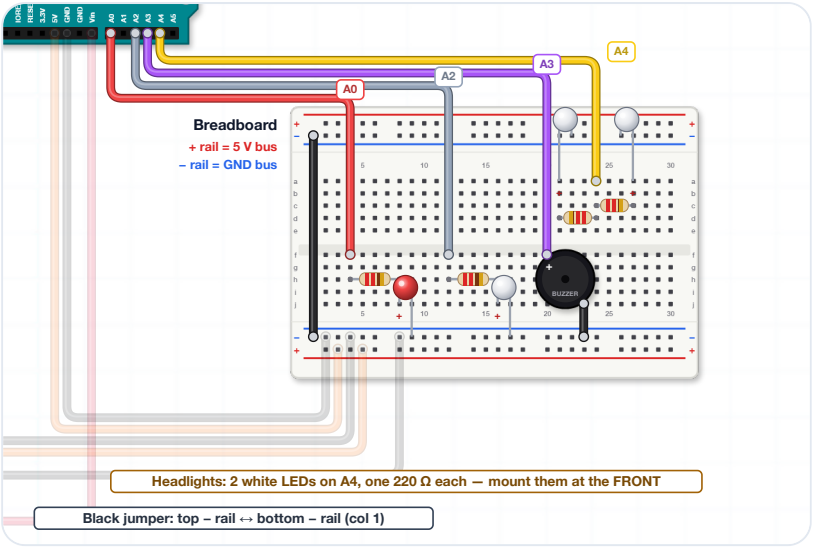
The HC-SR04 is mounted at the **rear** of the car, facing backwards (Module 03 shows the height).

1. HC-SR04 **TRIG** → UNO **D13**.
2. HC-SR04 **ECHO** → UNO **D12**.
3. **VCC** → + rail, **GND** → – rail.
4. Test: `tests/02_ultrasonic_test`.

	From	To	Note
	HC-SR04 TRIG	D13	UNO sends the ping
	HC-SR04 ECHO	D12	UNO measures the echo
	HC-SR04 VCC	+ rail	5 V
	HC-SR04 GND	– rail	

6 Lights + buzzer

A0 red rear LED · A2 white reverse LED · A3 buzzer · A4 two headlights



These parts sit on the breadboard. The analog pins A0–A4 work as normal digital outputs. The rear parts use the **bottom half** of the breadboard, the headlights the **top half**.

1. A0 wire → hole **4f**. 220 Ω resistor from **4h** to **8h**. Red LED: long leg (+) in **8j**, short leg in the **– rail**.
2. A2 wire → **12f**. 220 Ω from **12h** to **16h**. White LED: long leg in **16j**, short leg in the – rail.
3. A3 wire → **20f**. Buzzer **+** leg in **20h**, – leg in **23h**, short black jumper from **23j** to the – rail.
4. **Headlights:** A4 wire → **24a**. 220 Ω from **24c** to **27c**, white LED long leg in **27a**, short leg in the **top – rail**. Second 220 Ω from **24d** to **21d**, white LED long leg in **21a**, short leg in the top – rail.
5. Black jumper from the **top – rail** to the **bottom – rail** (column 1), so the top rail is GND too.
6. On the car the headlights sit at the **front**: put each LED on two female-male jumpers instead of in the breadboard.
7. Test: `tests/03_buzzer_led_test`.

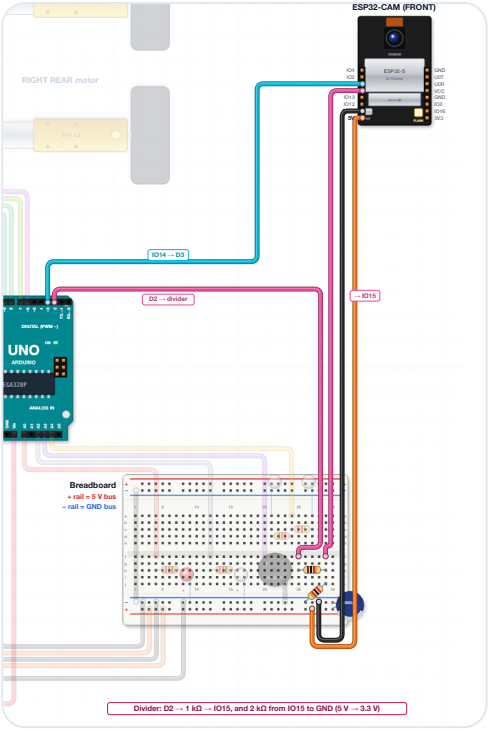
	From	To	Note
	`A0`	220 Ω → red LED (+)	rear warning light
	`A2`	220 Ω → white LED (+)	reverse light
	`A3`	Buzzer +	buzzer – to – rail
	`A4`	2 × (220 Ω → white LED +)	headlights, one resistor each
	Top – rail	Bottom – rail	bridge jumper

LED tips

The **longer leg** is + (anode); an LED wired backwards simply stays off. Each headlight has **its own 220 Ω**: with one shared resistor the brighter LED would take most of the current. Two headlights on A4 (≈ 26 mA) are fine for one UNO pin.

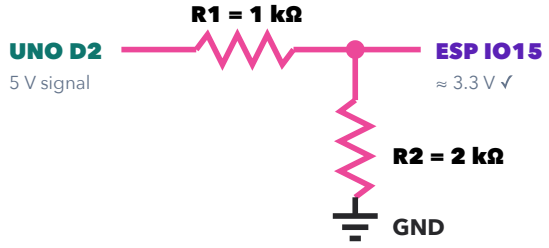
7 ESP32-CAM

IO14 → D3 · D2 → divider → IO15 · 5V / GND · 2200 µF capacitor



Why a voltage divider?

The UNO speaks 5 V, but ESP32 pins only accept 3.3 V. Two resistors make 5 V × 2k / (1k + 2k) = **3.33 V**. The other direction (3.3 V into the UNO) needs no divider: the UNO reads 3.3 V as HIGH.



1. Divider: UNO **D2** wire → hole **25f**. **1 kΩ** from **25h** to **29h**. **2 kΩ** from **29j** to the − rail.
2. ESP32-CAM **IO15** → hole **29f** (middle of the divider).
3. ESP32-CAM **IO14** → UNO **D3** (direct wire).
4. ESP32-CAM **5V** → + rail, **GND** → − rail (use the 5V pin, **not** 3V3).
5. **2200 µF capacitor** across the rails next to the ESP wires: the leg on the side with the stripe (−) goes in the − rail.
6. Leave **U0T, U0R and IO0** free: they are for programming.

	From	To	Note
	`IO14` (TX)	`D3` (RX)	direct wire
	`D2` (TX)	1 kΩ → `IO15` (RX)	2 kΩ from IO15 to GND
	`5V`	+ rail	not 3V3!
	`GND`	− rail	

MODULE 02

Full pin map & final check

Arduino UNO

Pin	Connected to
D2	→ 1 kΩ → ESP32 IO15 (2 kΩ to GND)
D3	← ESP32 IO14
D6 ~	L298N ENB
D7	L298N IN4
D8	L298N IN3
D9	L298N IN2
D10	L298N IN1
D11 ~	L298N ENA
D12	HC-SR04 ECHO
D13	HC-SR04 TRIG
A0	Red rear LED (220 Ω)
A1	Servo (optional)
A2	White reverse LED (220 Ω)
A3	Active buzzer +
A4	2 headlights (220 Ω each)
VIN	L298N +12V (7.4 V)
5V / GND	+ rail / − rail
D0 / D1	free (USB)

ESP32-CAM

Pin	Connected to
5V	+ rail
GND	− rail
IO14	TX → UNO D3
IO15	RX ← divider ← UNO D2
IO4	on-board flash LED = cam light
IO33	on-board red LED = status
U0T U0R IO0	programming only

L298N

Terminal	Connected to
OUT1/OUT2	left motors (parallel)
OUT3/OUT4	right motors (parallel)
+12V	switch (battery +) and UNO VIN
GND	battery − and breadboard − rail
+5V	not connected
ENA…ENB	UNO D11, D10, D9, D8, D7, D6

Final check before the first power-on

- [ ] Battery polarity checked, 5 V rail reads 4.9–5.1 V
- [ ] ENA/ENB jumpers removed, 5V-EN jumper on
- [ ] No 7.4 V wire touches a 5 V or signal pin
- [ ] All grounds connected (check with the multimeter)
- [ ] Divider in place between D2 and IO15
- [ ] ESP32-CAM on 5V (not 3V3)
- [ ] 2200 µF capacitor polarity correct
- [ ] Wheels off the ground for the first test

Wired by (student) _____ Checked by (teacher) _____ Date _____

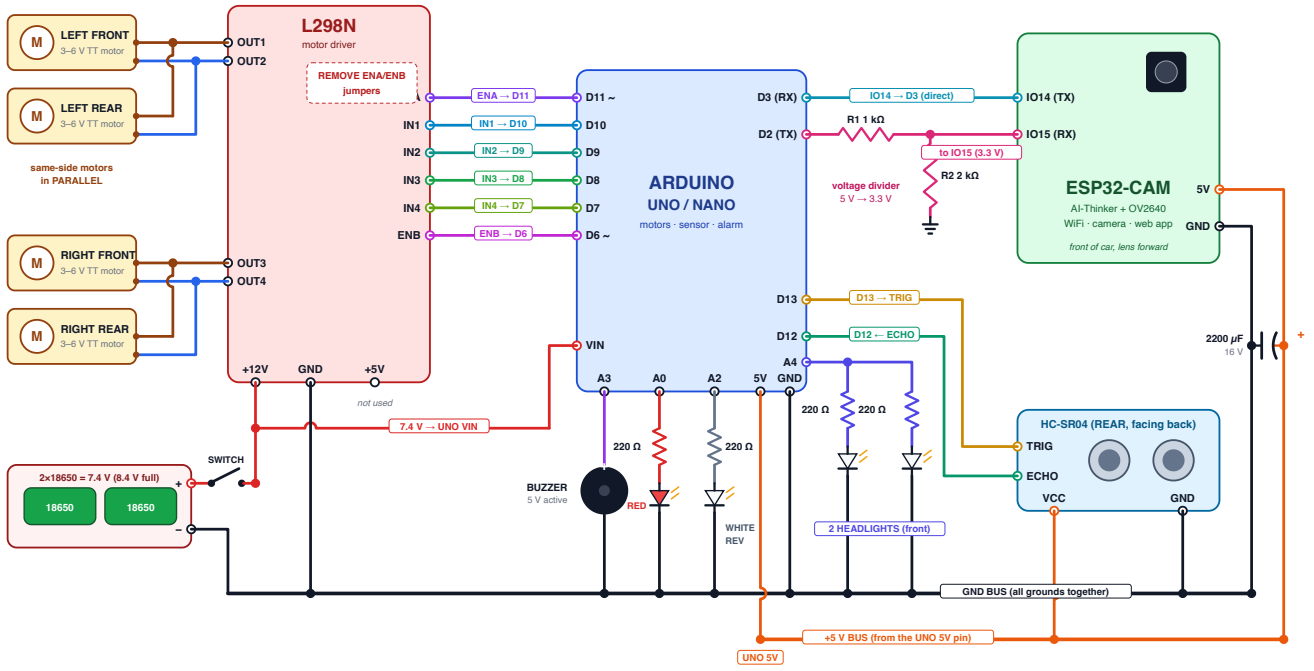
MODULE 02

Schematic view

The same circuit drawn as a schematic. It is useful for understanding and for finding faults.

ROBOT CAR – SCHEMATIC WIRING DIAGRAM

4WD chassis · L298N · Arduino UNO/Nano · ESP32-CAM · HC-SR04 (rear) · buzzer + rear LEDs · 2 headlights · 2×18650 (no buck converter: the UNO makes the 5 V)



POWER — no buck converter needed

- Battery 7.4 V → switch → L298N +12V. A 2nd wire in the same +12V terminal → UNO VIN.
- The UNO regulator makes the 5 V: UNO 5V pin → 5 V bus (ESP32-CAM, HC-SR04, buzzer, LEDs).
- 2200 µF 16 V capacitor right at the ESP32-CAM 5V/GND pins (+ leg to 5V).
- L298N: keep the 5V-EN jumper ON, remove ENA/ENB jumpers, leave its +5V pin unused.
- Do not add a servo to this 5 V bus (the UNO regulator is too small for it).

WIRE COLOURS

Battery +7.4 V · +5 V · GND · Motor wires · UART ESP→UNO
UART UNO→ESP · TRIG · ECHO · Buzzer · Headlights

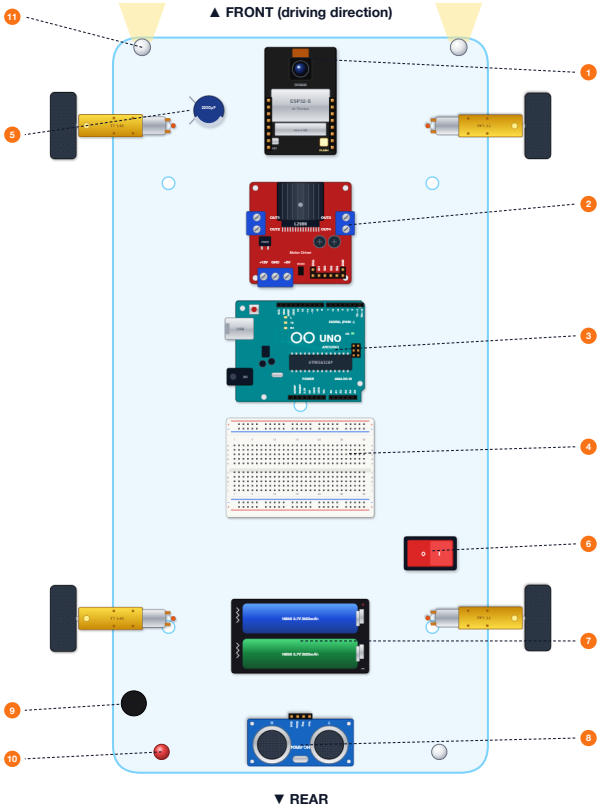
- Wires crossing with a small hop are NOT connected. A black dot = connected.
- **ALL grounds must be common. Never feed 7.4 V into the ESP32-CAM or UNO 5V pin.**

MODULE 03

03

Assembly Guide

Where every part goes on the chassis, how to mount it, and the mechanical safety check.



Name _____ Class / Group _____ Date _____

4WD · Arduino UNO · ESP32-CAM
Edition 2026

MODULE 03

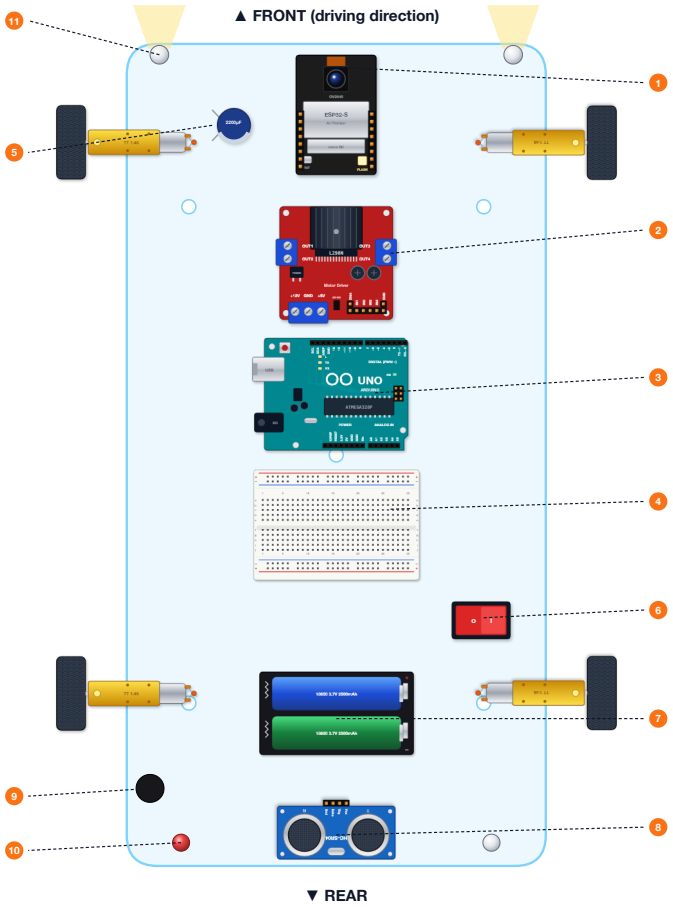
1 Mechanical assembly

A neat mechanical build makes wiring easier and the car more reliable. Motors and the battery go on the **bottom plate**; the electronics go on the **top plate**. Follow the layout below.

LEARNING OBJECTIVES · ABOUT 90 MIN

By the end of this module you can:

- Prepare motors with noise-suppression capacitors
- Place every part where it works best (camera, sensor, battery, antenna)
- Mount the rear sensor at the right height and angle
- Do a mechanical safety check before power-on



Top view: where each part goes (front at the top)

Layout legend

1. ESP32-CAM: front edge, lens forward, tilted up 5–10°
2. L298N motor driver: bottom plate, centre, heatsink free
3. Arduino UNO: top plate, centre
4. Breadboard: 5 V / GND rails, resistors, LEDs, buzzer
5. 2200 µF capacitor next to the ESP32-CAM 5V
6. ON/OFF switch: on the side, easy to reach
7. 2×18650 battery: low and towards the rear
8. HC-SR04: rear edge, facing back, 8–12 cm high
9. Buzzer: anywhere at the rear
10. Rear: red LED (A0) left, white reverse LED (A2) right
11. 2 white headlights (A4) at the front corners

WiFi antenna

Keep the ESP32-CAM antenna end (the zig-zag gold trace on the module) away from metal, motors and the L298N. Otherwise the WiFi range drops.

MODULE 03

2 Step by step

Step 1: Prepare the motors

1. Solder about 15 cm of wire to both tabs of each motor (red + black or two colours).
2. Solder a **100 nF ceramic capacitor** across the two tabs of each motor. This reduces electrical noise that disturbs the camera and WiFi.
3. Twist each motor's two wires together.
4. Mount the motors with the acrylic brackets so all **tabs face inwards**.

Step 2: Bottom plate

1. Fix the **L298N** in the centre, motor terminals facing the motors. Leave air around the heatsink.
2. Place the **battery holder** towards the rear. Mount the **switch** on the side.
3. Connect the motor wires to OUT1–OUT4 (Module 02, step 2).

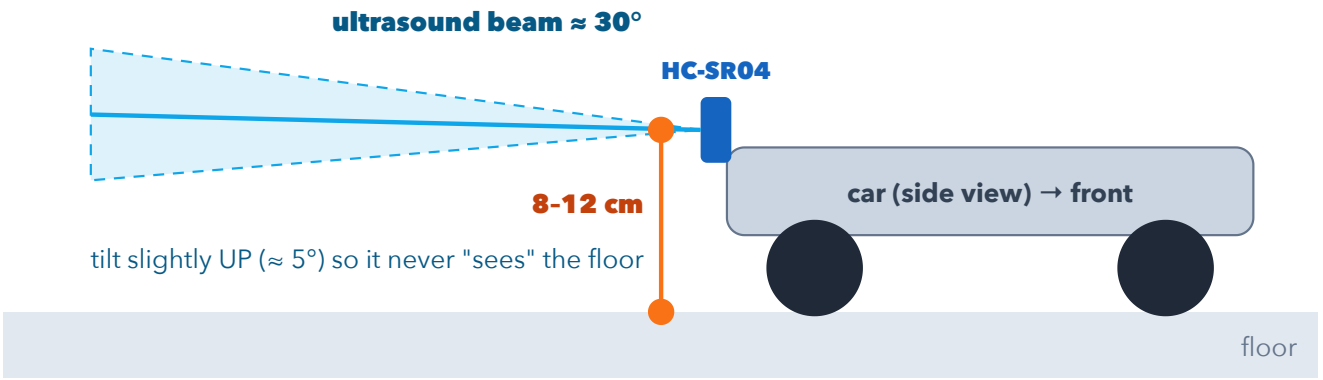
Step 3: Spacers and top plate

1. Screw in the brass spacers and mount the top plate.
2. Pass the control wires (ENA…ENB) and the power wires up through the centre opening.
3. Fix the **Arduino UNO** (nylon standoffs or foam tape) and stick the **breadboard** next to it.

Step 4: Front, the camera and the headlights

1. Mount the **ESP32-CAM at the very front**, camera looking forward, tilted up 5–10° (3D-printed holder, L-bracket or foam tape).
2. Make sure the camera ribbon cable is fully inserted and the black latch is closed.
3. Fix the **two white headlights** at the front corners, pointing forward (5 mm LED holders or hot glue). Do not cover the camera lens.
4. Run the headlight wires back to the breadboard along the side of the plate (long leg = + towards its 220 Ω resistor).

Step 5: Rear, sensor, buzzer and LEDs



Side view: correct sensor height and angle

1. Mount the **HC-SR04 on the rear edge, facing straight back**, 8–12 cm above the floor, tilted up slightly.
2. Nothing (wires, screws, wheels) may stick out in front of its two "eyes".
3. Fix the red LED at the left rear corner (like a brake light), the white reverse LED at the right rear corner, and the buzzer at the rear.

> **Sensor too low or pointing down?**
> It will detect the floor and beep, or block reversing, all the time. Raise it and tilt it up.

Step 6: Cable management

- Keep the power wires short. Use zip ties to keep wires away from the wheels.
- Route signal wires away from motor wires where possible.
- Battery low and centred or slightly to the rear: a skid-steer car turns better that way.

Mechanical check

- [ ] All 4 wheels turn freely by hand
- [ ] No wire can touch a wheel
- [ ] Battery holder is firmly fixed
- [ ] Camera faces forward and is tilted up a little
- [ ] Sensor faces back, 8–12 cm high, nothing in front of it
- [ ] Headlights at the front, pointing forward
- [ ] Switch is reachable

Built by (student) _____

Checked by (teacher) _____

ROBOT CAR PROJECT

MODULE 04

04

Software & App Guide

Install the tools, upload both boards, connect the phone and master every control.



Name _____ Class / Group _____ Date _____

4WD · Arduino UNO · ESP32-CAM
Edition 2026

MODULE 04

1 Software & uploading

The car runs two programs: one on the **Arduino UNO** (motors, sensor, alarm) and one on the **ESP32-CAM** (WiFi, camera, phone app). Both are uploaded with the free **Arduino IDE 2**. No extra libraries are needed.

LEARNING OBJECTIVES · ABOUT 60 MIN

By the end of this module you can:

- Install the Arduino IDE and the ESP32 board package
- Upload code to an Arduino UNO and to an ESP32-CAM
- Read the Serial Monitor to check that a board works
- Connect a phone and use every feature of the app

1 • Install the software

1. Download and install **Arduino IDE 2** from `arduino.cc/en/software`.
2. Open **File → Preferences**. In *Additional boards manager URLs* paste:

```
https://espressif.github.io/arduino-esp32/package_esp32_index.json
```

3. Open **Tools → Board → Boards Manager**, search **esp32** and install **"esp32 by Espressif Systems"** (version 3.x).
4. Open the project folder. The code is in `firmware/uno_controller` and `firmware/esp32cam_robot`.

2 • Board settings

Setting	Arduino UNO	ESP32-CAM
Sketch	`uno_controller.ino`	`esp32cam_robot.ino`
Tools → Board	Arduino AVR Boards → **Arduino Uno**	esp32 → **AI Thinker ESP32-CAM**
Partition scheme	–	**Huge APP (3MB No OTA)**
PSRAM	–	Enabled (if the menu exists)
Port	the UNO's USB port	the MB board's port (CH340)

Arduino Nano or a clone?
Choose **Arduino Nano** and, if the upload fails, set **Processor → ATmega328P (Old Bootloader)**.

3 • Upload the Arduino UNO

1. Open `uno_controller.ino` and select the board and port.
2. Click **Upload** (→). You can leave all wires connected.
3. After a reset you hear a **short beep** and the **headlights blink**: the UNO is running.

4 • Upload the ESP32-CAM

Option A: ESP32-CAM-MB board EASY

1. Unplug the ESP32-CAM from the car and press it onto the MB board.
2. Connect the USB cable, select the port, click **Upload**.
3. Stuck at `Connecting....`? Hold **IO0**, tap **RST**, release IO0.
4. When it is done, press **RST** to start the program.

Option B: FTDI adapter

FTDI	ESP32-CAM
5V (jumper on 5 V)	5V
GND	GND
TX	U0R
RX	U0T
–	**IO0** → **GND** while uploading

Press RST, upload, then **remove the IO0-GND wire** and press RST again.

Check the Serial Monitor (115200 baud)

```
=== ROBOT CAR ESP32-CAM ===
WiFi network "RobotCar" / password "12345678"
Open  http://192.168.4.1  on your phone
Ready.
```

"Camera init FAILED"
The ribbon cable is not fully inserted (open the latch, push it in, close it) or the wrong board type is selected.

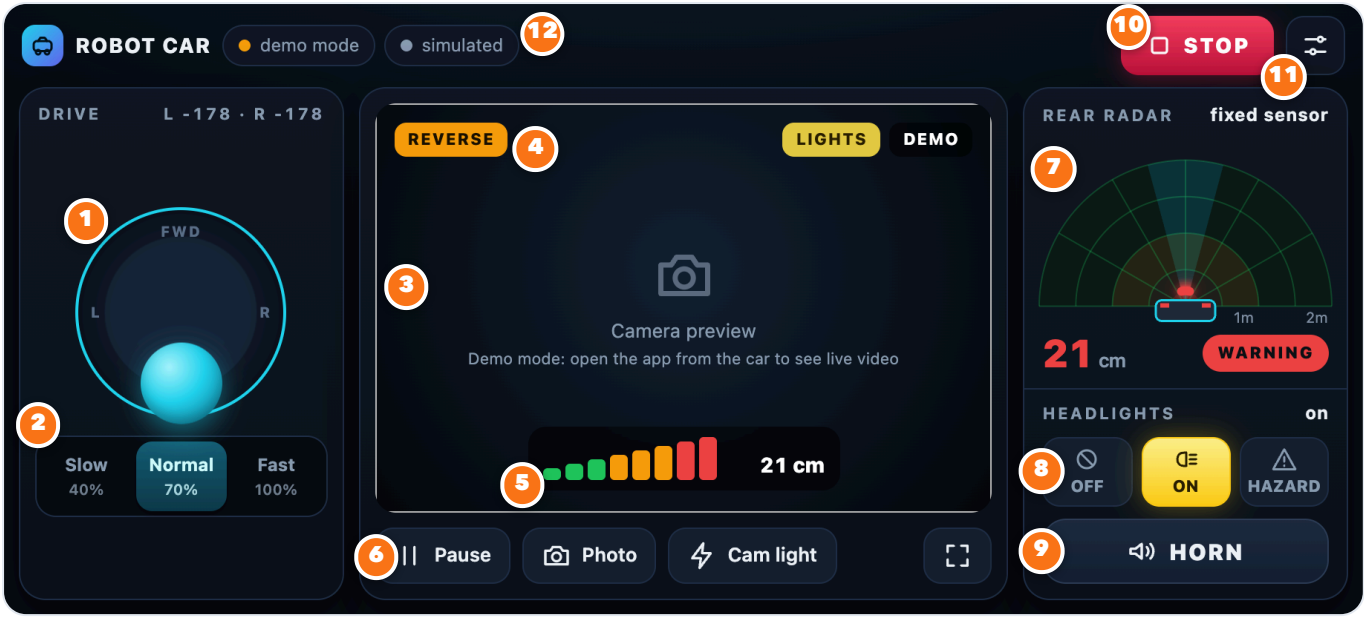
5 • Connect the phone

1. Switch on the car. The small red LED on the back of the ESP32-CAM **blinks**: it is waiting for a phone.
2. On the phone, open WiFi settings → network **RobotCar**, password **12345678**. If the phone warns "no internet", choose **stay connected**.
3. Open the browser at **http://192.168.4.1**. The chips show **connected** and **car ok**.
4. Turn the phone sideways and tap ⛶ for full screen. Tip: **Add to Home screen** so it opens like an app.
5. The first time, a short **tips** screen explains the controls. Tap **Let's drive**.
6. Optional: on a PC joined to the same WiFi, open **http://192.168.4.1/view** for the classroom viewer (section 3).

Built-in safety
Release the joystick or press the red **STOP**: the car stops. Lock the phone, switch app or lose WiFi: it stops within 0.6 s. It never reverses closer than the auto-brake distance (forward still works).
Keyboard (app on a laptop): W A S D or arrows = drive · Space = horn · L = headlights · Esc = stop.

MODULE 04

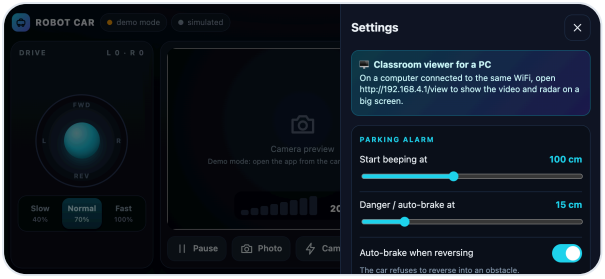
2 Using the app



The phone app in landscape: reversing towards an obstacle at 21 cm, headlights on

#	Element	What it does
1	**Joystick**	Drag up/down = forward/back, left/right = turn. Release = stop. (A D-pad mode is in the settings.)
2	Speed gears	**Slow** 40 % · **Normal** 70 % · **Fast** 100 %. Start with Slow
3	**Live video**	From the front camera. A flashing **red border** means an obstacle is close behind while reversing
4	Status tags	**REVERSE** while backing up, **AUTO-BRAKE** when the car refuses to go back, **LIGHTS** when the headlights are on
5	Parking bar	More bars lit = closer, green → amber → red. Shows the distance in cm
6	Video buttons	**Pause** the video · save a **Photo** on the phone · **Cam light** = the camera's own flash LED · ⛶ full screen
7	**Rear radar**	Distance behind the car: green, amber or red zone, beam cone, big distance number and zone name
8	**Headlights**	**OFF** · **ON** · **HAZARD** (headlights and red rear LED blink together)
9	**Horn**	Hold to sound the car's buzzer
10	**STOP**	Stops the car at once
11	Settings	See the next page
12	Status chips	**connected** to the car and **car ok** (the UNO answers). Amber or red = check the WiFi

Settings



Settings sheet (tap the sliders icon)



Portrait mode works too

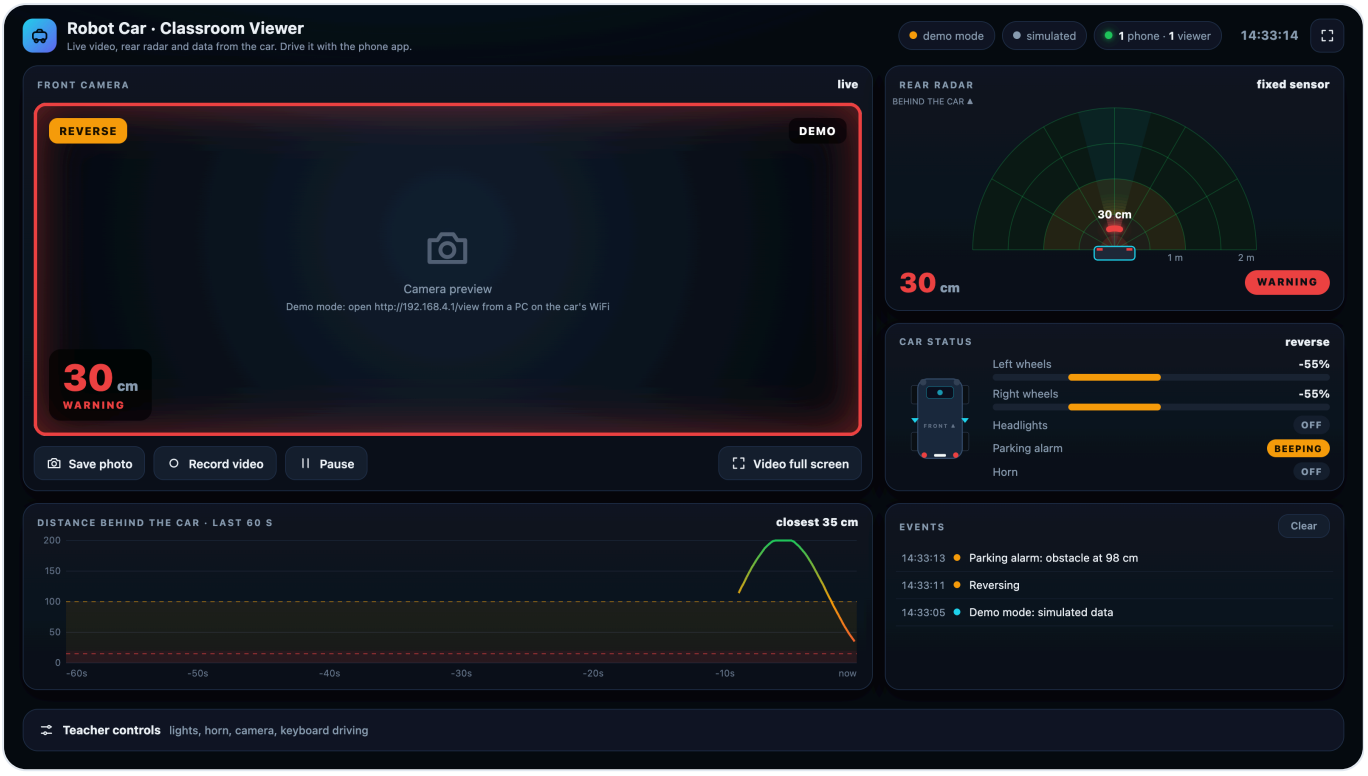
Setting	Meaning
Parking alarm	
Start beeping at	Beeping starts below this distance (default 100 cm)
Danger / auto-brake at	Continuous tone + brake below this (default 15 cm)
Auto-brake when reversing	Block reversing near obstacles
Alarm also when not reversing	Beep in any direction
Mute the car buzzer	The LED still flashes
Beep on this phone / Vibrate	The phone copies the alarm
Driving	
Controls	Joystick or D-pad
Exact speed	Fine-tune the speed gear (30–100 %)
Turn sensitivity	How sharp the car turns
Camera	
Resolution / picture quality	VGA = default. QVGA = smoothest video
Cam light brightness	Brightness of the camera's flash LED
Flip / Mirror	If the image is upside down or reversed
Radar & connection	
Radar range	Distance shown at the edge of the radar
Car address	Leave 192.168.4.1 unless you changed the WiFi mode
Show the tips again	Opens the first-time tips screen

MODULE 04

3 Classroom viewer on a PC

The car serves a **second page** made for a computer or a projector. One student drives with the phone while the class watches the video, the radar and live data on the big screen. Nothing needs to be installed.

1. Connect the PC to the WiFi **RobotCar** (password **12345678**).
2. Open **http://192.168.4.1/view** in Chrome, Edge or Firefox. (The phone app shows the same link in its settings, and a banner on a PC.)
3. Press **F11** or the full-screen button for the projector.



The classroom viewer (here in demo mode, reversing at 28 cm)

Part of the page	What it shows or does
Front camera	Big live video with the REVERSE / AUTO-BRAKE tags and a large distance number while reversing
Save photo · Record video	Photo as JPG with a data strip (time, distance, speeds). Video as a WebM file with the distance drawn on it
Rear radar	The same radar as on the phone, larger
Car status	Picture of the car: headlight beams, rear and reverse lights, wheel arrows, left/right motor bars, alarm and horn
Distance chart	The distance behind the car for the last 60 s, with the warning and danger lines. Closest value shown
Events	Log with time: reversing, alarms, auto-brake, lights, connection
Teacher controls	Headlights, horn, cam light, radar range, and **keyboard driving** (W A S D / arrows, Space = horn, Esc = stop)

How many screens?

Up to **3 video screens** at once (for example the phone and two PCs). All screens get the radar and data. Close unused tabs if a screen says “too many viewers”.

Exam and report ideas

Record a video of a parking manoeuvre, then use the chart to read the closest distance and the moment the auto-brake stopped the car.

Customising the car

File · setting	Default	Change it to...
`config.h` · AP_SSID / AP_PASSWORD	RobotCar / 12345678	give every group its own WiFi name
`config.h` · STA_SSID	empty	join a school WiFi or phone hotspot instead
`config.h` · CAM_VFLIP	0	1 if the camera is mounted upside down
`uno_controller.ino` · MIN_PWM	90	raise if the car doesn't move at low speed
`uno_controller.ino` · INVERT_LEFT/RIGHT	false	flip a side that drives backwards
`uno_controller.ino` · USE_SERVO	0	1 = scanning radar with an SG90 servo on A1

Several cars in one classroom

Give each car a different `AP_SSID` (RobotCar1, RobotCar2 …) and a different `AP_CHANNEL` (1, 6, 11) so they don't interfere.

Editing the pages

The phone app is `web/index.html`, the PC viewer `web/view.html`. Open them on a computer to try them in **demo mode**. Then run `python3 tools/build_ui.py` and upload the ESP32-CAM again.

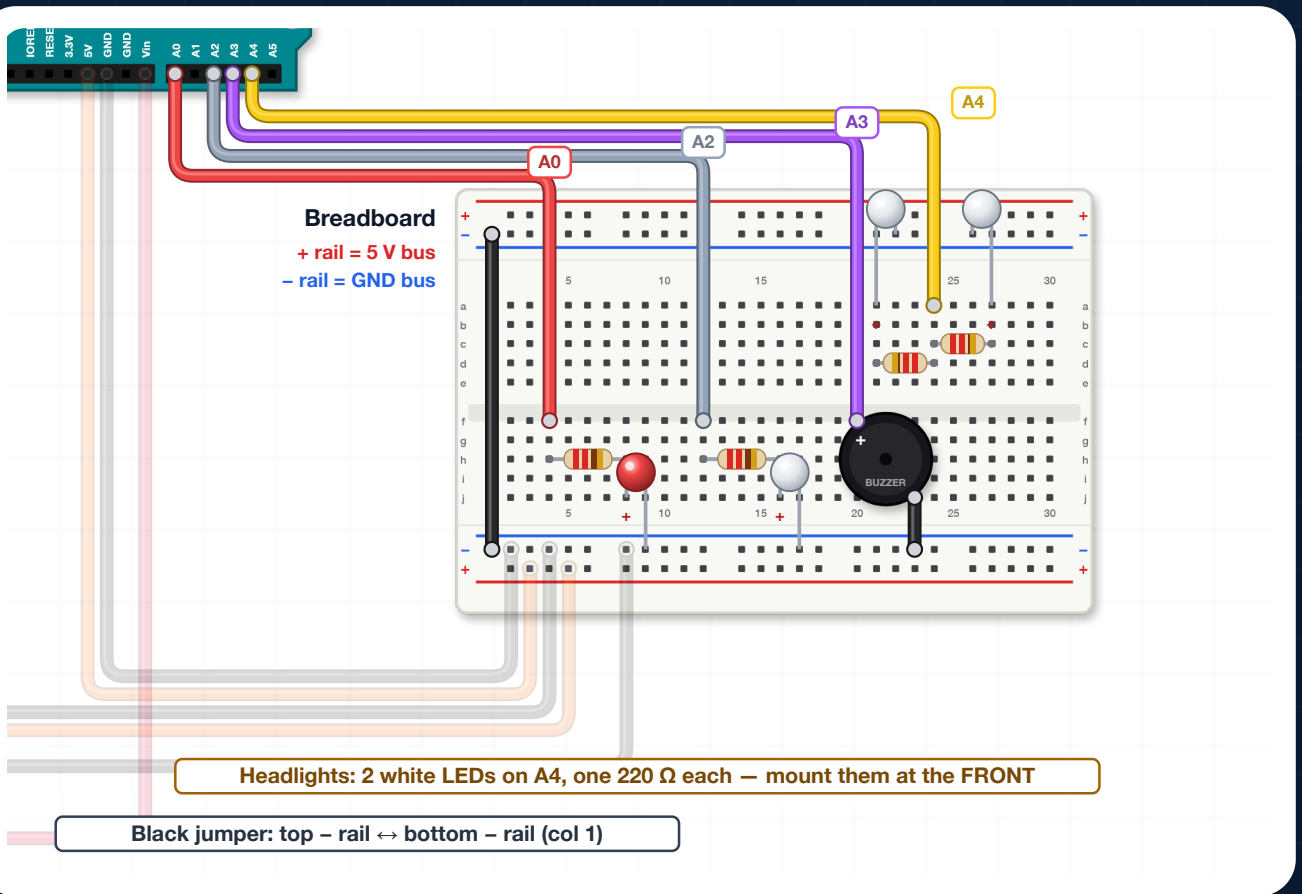
ROBOT CAR PROJECT

MODULE 05

05

Testing & Troubleshooting

Test one part at a time, record the results, and fix the common problems yourself.



Name _____ Class / Group _____ Date _____

4WD · Arduino UNO · ESP32-CAM
Edition 2026

MODULE 05

1 Testing, step by step

Never test everything at once. Upload the small **test sketches** in `tests/` one by one: when something fails, you know exactly which part to look at. Fill in this record as you go.

LEARNING OBJECTIVES · ABOUT 60 MIN

By the end of this module you can:

- Test a system part by part (unit tests before system tests)
- Use the Serial Monitor and a multimeter to find faults
- Fix the most common problems on your own

Lab record: bring-up tests

#	Test	How	Expected result	Pass
1	Power	Battery on, multimeter on the breadboard + and − rails (= UNO 5V)	4.9–5.1 V	☐
2	Motors	Upload `01_motor_test`, wheels in the air	All forward → all back → spin left → spin right	☐
3	Sensor	Upload `02_ultrasonic_test`, Serial Monitor 115200	Distance in cm follows your hand	☐
4	Lights + buzzer	Upload `03_buzzer_led_test`	Headlights on, 4 hazard blinks, then beeps get faster and continuous. Red LED in sync	☐
5	Bench alarm	Upload `06_parking_alarm_bench`, hand towards sensor	SAFE → WARN (faster beeps) → STOP! (continuous)	☐
6	Camera only	Upload `05_esp32cam_test`, join WiFi “CamTest”, open 192.168.4.1	Live video, Photo works, flash LED dims	☐
7	ESP32-CAM	Upload `esp32cam_robot`, Serial Monitor	“Ready.” and WiFi “RobotCar” visible	☐
8	Serial link	Upload `04_link_test` on the UNO, open the app	Radar shows 20→200 cm. Serial shows `M,…` lines	☐
9	Full car	Upload `uno_controller`	Drive with the phone	☐

Acceptance test: the finished car

#	Test	How	Expected result	Pass
1	Drive	Forward 2 m, turn left and right	Smooth, straight, responds immediately	☐
2	Video	Watch the camera while driving	Clear image, little delay	☐
3	Parking alarm	Reverse slowly towards a wall	Beeping starts at about 100 cm and speeds up, continuous near 20 cm	☐

#	Test	How	Expected result	Pass
4	Auto-brake	Keep reversing	Stops at about 15 cm, AUTO-BRAKE tag on the phone. Forward still works	☐
5	Radar	Move a box behind the car	Radar distance and colour change	☐
6	Headlights	Tap ON, then HAZARD in the app	Both front LEDs light. HAZARD blinks them with the red rear LED	☐
7	PC viewer	Open `192.168.4.1/view` on a PC while driving	Video, radar, car status and chart update live	☐
8	Fail-safe	Drive, then turn phone WiFi off	Car stops within 1 s	☐

Tested by (student) _____ Verified by (teacher) _____ Date _____

MODULE 05

2 Troubleshooting

Power & resets

Symptom	Likely cause	Fix
ESP32-CAM reboots when the motors start (`Brownout detector was triggered`)	5 V dips	Check the 2200 µF capacitor (polarity, close to the ESP). ESP on the UNO 5V rail, not the L298N's 5V. Short power wires, good charged 18650s. Try the Slow gear
Everything dies at full speed	Weak or fake batteries	Better cells, charge them, lower the speed slider
L298N gets hot	Normal up to ~60 °C	Keep the heatsink free. Don't stall the motors
UNO regulator (next to the DC jack) very hot	Too much on the 5 V rail	Warm is normal. Never put a servo or motor on the 5 V rail
Nothing turns on with the battery	VIN wire missing or loose	Check the second red wire L298N +12V → UNO VIN and the battery polarity

Motors

Symptom	Fix
Nothing moves, but the app shows L/R values	ENA/ENB jumpers still on? L298N GND connected? Battery on? Chip says “car not responding” → serial link
One side drives backwards	Swap that side's OUT wires, or set `INVERT_LEFT/RIGHT = true`
One motor of a pair fights the other	Swap only that motor's two wires
Turns left when you push right	Swap the left and right OUT pairs (OUT1/2 ↔ OUT3/4)
Hums but doesn't move at low speed	Raise `MIN_PWM` (110–130)
Won't reverse even with nothing behind	Sensor sees the floor or the chassis: mount it higher, tilt it up. Or turn auto-brake off

Camera & WiFi

Symptom	Fix
`Camera init FAILED`	Re-seat the ribbon cable, close the latch. Board = AI Thinker ESP32-CAM
No “RobotCar” network	Check the Serial Monitor. Maybe STA mode is set in `config.h`
Phone switches back to mobile data	Turn mobile data off, or choose “stay connected” when the phone says “no internet”
Video stutters	Settings → QVGA, quality 15–20. Keep the camera away from the motors
Video OK but controls “offline”	Reload the page. Use `http://`, not https. Disable data-saver mode

Sensor & alarm

Symptom	Fix
Radar: “waiting for sensor…”	UNO not sending: check D2 → 1k → IO15, IO14 → D3, common GND, UNO powered
Distance always “>400”	TRIG/ECHO swapped, no 5 V on VCC, or nothing within 4 m
Buzzer silent, LED works	Passive buzzer (needs an active one), wrong polarity, or “Mute the car buzzer” is on

Lights & PC viewer

Symptom	Fix
No headlight lights up	A4 wire in row 24 (top half)? Top − rail bridged to the bottom − rail? Does the LIGHTS tag appear in the app?
Only one headlight works	That LED is backwards (long leg = +), or its resistor is not in the same column as the LED's long leg
PC viewer: “too many viewers”	Max 3 video screens at once. Close other tabs or the photo page
PC viewer: no data, only demo	Open `http://192.168.4.1/view`, not the file from the project folder. PC on the RobotCar WiFi

Multimeter checks

Measure between	Should read
Battery + and −	7.0–8.4 V (recharge below 6.8 V)
UNO 5V (+ rail) and GND	4.9–5.1 V
ESP32-CAM 5V and GND while driving	above 4.75 V
ESP32 IO15 and GND, car idle	≈ 3.3 V (divider OK). **Never 5 V**

MODULE 06

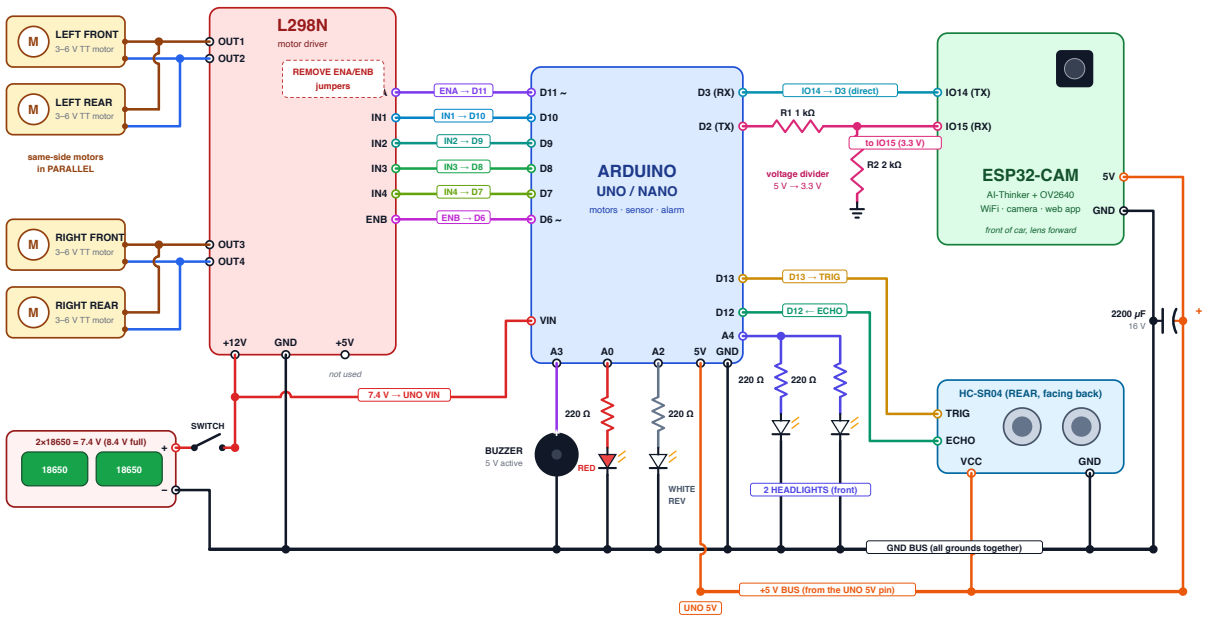
06

Technical Reference

Protocol, safety rules, steering and alarm maths, code structure and extension ideas.

ROBOT CAR – SCHEMATIC WIRING DIAGRAM

4WD chassis · L298N · Arduino UNO/Nano · ESP32-CAM · HC-SR04 (rear) · buzzer + rear LEDs · 2 headlights · 2×18650 (no buck converter: the UNO makes the 5 V)



POWER — no buck converter needed

- Battery 7.4 V → switch → L298N +12V. A 2nd wire in the same +12V terminal → UNO VIN.
- The UNO regulator makes the 5 V: UNO 5V pin → 5 V bus (ESP32-CAM, HC-SR04, buzzer, LEDs).
- 2200 µF 16 V capacitor right at the ESP32-CAM 5V/GND pins (+ leg to 5V).
- L298N: keep the 5V-EN jumper ON, remove ENA/ENB jumpers, leave its +5V pin unused.
- Do not add a servo to this 5 V bus (the UNO regulator is too small for it).

WIRE COLOURS

Battery +7.4 V · +5 V · GND · Motor wires · UART ESP→UNO · UART UNO→ESP · TRIG · ECHO · Buzzer · Headlights

- Wires crossing with a small hop are NOT connected. A black dot = connected.
- **ALL grounds must be common. Never feed 7.4 V into the ESP32-CAM or UNO 5V pin.**

Name _____ Class / Group _____ Date _____

MODULE 06

1 How the system works

This reference explains what happens inside the code: the messages between the phone, the ESP32-CAM and the Arduino, the safety rules, and the maths behind driving and the parking alarm. Use it for oral exams, reports and your own improvements.

LEARNING OBJECTIVES · ABOUT 45 MIN

By the end of this module you can:

- Describe the data flow between phone, ESP32-CAM and Arduino
- Explain a simple text protocol with a checksum
- Explain differential (tank) steering and the parking-alarm timing
- Plan an extension of the project

Data flow

```
Phone browser ──WebSocket ws://192.168.4.1/ws (text)──► ESP32-CAM ──UART 38400──► Arduino UNO
Phone browser ◄──────────────────────────────────────── ESP32-CAM ◄──────────────── Arduino UNO
PC  /view     ◄── same WebSocket (data) ──────────────── ESP32-CAM
Phone + PC <img> ◄── MJPEG http://192.168.4.1:81/stream — ESP32-CAM  (max 3 screens)
```

Messages

Phone → ESP32-CAM (WebSocket)

Message	Meaning	Handled by
`M,<left>,<right>`	Motor speeds −255…255, sent every 100 ms	forwarded to UNO
`H,0|1`	Horn off/on	forwarded
`C,warn,stop,brake,always,mute`	Alarm settings (cm, cm, 0/1, 0/1, 0/1)	forwarded (repeated every 3 s)
`L,0|1|2`	Headlights off / on / hazard	forwarded (repeated every 3 s)
`K,0–255`	Cam light: brightness of the ESP32-CAM flash LED	ESP32
`F,n` / `Q,n`	Camera frame size / JPEG quality	ESP32
`V,0|1` / `X,0|1`	Flip / mirror image	ESP32
`R,phone` / `R,view`	The page says what it is: a driving phone or a PC viewer (for the counters)	ESP32

ESP32-CAM → UNO (serial, with checksum)

The same `M`, `H`, `C`, `L` and `S` lines, plus `*XX` = the XOR of every character before the `*`, in hex. Example: `M,0,0*4D`. The UNO's software serial port can lose bytes while it is transmitting. A damaged line fails the checksum and is ignored, and the next one arrives 150 ms later.

UNO → ESP32-CAM → phone

Message	Meaning
`D,<angle>,<cm>,<flags>`	Every 60 ms. angle = 90 (fixed sensor) or 30–150 (servo). cm = 0 means nothing in range. flags (add them up): 1 reversing · 2 alarm · 4 auto-brake · 8 horn · 16 headlights lit · 32 hazard
`T,l,r,light,horn,flash`	ESP32 → every screen, every 250 ms: motor speeds, headlight mode, horn and cam light, so all screens stay in sync
`I,rssi,phones,uno_ok,viewers`	ESP32 status every 2 s: WiFi signal, number of phones, UNO answering, number of PC viewers

Web addresses

Address	Returns
`http://192.168.4.1/`	the phone app
`/view`	the PC classroom viewer
`/status`	JSON with distance, flags, lights, camera settings
`/cmd?c=M,150,150`	send a command over plain HTTP (for other apps, e.g. MIT App Inventor)
`/capture`	one JPEG photo
`:81/stream`	live MJPEG video

MODULE 06

2 Inside the code

Safety rules (fail-safes)

Where	Rule
Phone	Sends stop when the joystick is released or the page is hidden
ESP32-CAM	No command from the phone for 500 ms → sends stop
Arduino	No command for 600 ms → motors off, horn off
Arduino	Reversing and distance ≤ stop distance and auto-brake on → backwards speed forced to 0
Arduino	Speed ramps 18 PWM steps every 10 ms → no current spikes that could reset the ESP32

Differential (tank) steering

The joystick gives **y** (forward/back) and **x** (left/right) between −1 and 1. Each side's speed is:

```
left  = y + x · turn        right = y − x · turn
divide both by max(1, |left|, |right|)   // keep within −1…1
PWM   = value × 255 × speed%
```

Push straight up and both sides are equal, so the car goes straight. Push right and the left side is faster, so the car turns right. Push fully right with y = 0 and the sides are opposite, so the car spins on the spot.

Parking alarm timing

```
if distance ≤ stop + 5 cm  →  continuous tone
else period = map(distance, stop+5 … warn, 120 ms … 900 ms)
     buzzer ON for the first half of each period
```

Closer means a shorter period, so the beeping is faster, exactly like a real car. The distance is the **median of the last 3 readings**, which removes single wrong echoes.

Files

File	Main parts
`firmware/uno_controller/uno_controller.ino`	`setSide()` motor driver · `rampTo()` soft start · `pingCm()`, `median3()` sensor · `updateAlarm()` · `handleLine()`, `checksumOk()` protocol
`firmware/esp32cam_robot/esp32cam_robot.ino`	`cameraInit()` · `streamHandler()` video · `wsHandler()`, `wsBroadcast()` WebSocket · `readUno()`, `sendToUno()`, `linkSend()`
`firmware/esp32cam_robot/config.h`	WiFi name and password, pins, camera options
`web/index.html`	Phone app: joystick, video, radar canvas, headlights, settings, demo mode

File	Main parts
`web/view.html`	PC viewer: video, photo/video recording, radar, car status, chart, events
`tests/01…06`	Small test sketches for each part (05 = camera only, 06 = bench parking alarm)

Ideas for your own improvements

Scanning radar
Mount the HC-SR04 on an SG90 servo (A1) and set `USE_SERVO 1`. The phone radar then shows a real sweep with blips.

Front collision stop
Add a second HC-SR04 at the front (free pins: D4, D5) and stop forward motion near obstacles.

Battery gauge
Measure the battery with a 10k/4.7k divider on A5 and show the voltage in the app with a new message `B,…`.

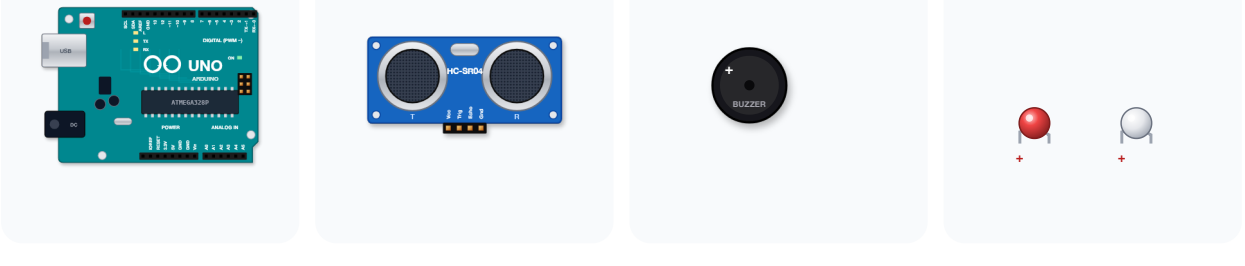
Your own app
Build an MIT App Inventor app that sends `/cmd?c=M,…` requests and shows `/status`.

MODULE 07

07

Warm-up Lessons

Use the free Learn Arduino website to learn every idea the car's code uses, then test it on a real UNO.



Name _____ Class / Group _____ Date _____

MODULE 07

1 Warm-up with the Learn Arduino website

learn-arduino-red.vercel.app is a free website with 16 interactive 3D lessons for the Arduino UNO. Each lesson shows the board and parts in 3D with the wires drawn. The **Overview, Pinout, Example and Specifications** tabs explain it, and you can **run the sketch in the browser** and see the result. Do the lessons marked **Must do** before you build the car: they teach every idea the car's UNO code uses.

LEARNING OBJECTIVES · ABOUT 90 MIN

By the end of this module you can:

- Explain digitalWrite, analogWrite (PWM), pulseIn and map() in your own words
- Read an HC-SR04 distance and turn it into zones with comparisons
- Move a sketch from the website's pins to the robot car's pins and test it on a real UNO

How to use the website

- Open it on a computer (Chrome or Edge) for the best 3D view. Drag to move, **Q / E** to rotate, scroll to zoom.
- Click a part in the 3D scene or a lesson's controls to change its values, then press ▶ **Run sketch** and watch the serial monitor.
- The code box is **read-only**. To change it, press **Copy code**, paste it into the Arduino IDE and upload it to a real UNO.
- The website only knows the **Arduino UNO**: there is no ESP32-CAM, L298N or WiFi. Those are taught in Modules 02 and 04 of this handbook.

Lesson map: website → robot car

#	Website lesson	You learn	Where the car uses it	
1	Meet the board	The UNO's pins: digital, PWM (~), analog, power	Module 02: every wire goes to one of these pins	**Must do**
2	Blink an LED	`pinMode`, `digitalWrite`, `delay`	Red rear LED `A0`, white reverse LED `A2`, headlights `A4`	**Must do**
3	Push a button	Digital input, pull-down resistor	Not on the car (idea: a start button)	Extra
4	Turn a knob	`analogRead`, `map()`	`map()` turns distance into beep speed	**Optional**
5	Mix colours	PWM with `analogWrite` 0–255	Motor speed on ENA `D11` / ENB `D6`	**Must do**
6	Sense light	Analog sensor (LDR)	Idea: LDR on `A5` switches the headlights on when it gets dark	Extra
7	Read a sensor	HC-SR04, `pulseIn()`, ÷58 = cm	Rear sensor: TRIG `D13`, ECHO `D12`	**Must do**
8	Make a sound	Buzzer, `tone()`	Alarm buzzer on `A3`, active type, no `tone()`	**Must do**
9	Mini piano	Arrays and loops	Idea: a melody for the horn	Extra

#	Website lesson	You learn	Where the car uses it	
10	Control a motor	PWM speed + transistor driver	The L298N does the same job and also reverses the motor	**Must do**
11	Sense heat	TMP36 temperature sensor	Not on the car	Extra
12	Sweep a servo	SG90 servo, `Servo.h`	Scanning radar option `USE_SERVO`, servo on `A1`	**Optional**
13	Spin the wheel	Stepper + ULN2003	Not on the car	Extra
14	Smart plant	Sensor + decision	Same idea as auto-brake: read → decide → act	Extra
15	Feel the force	Force sensor	Not on the car	Extra
16	Build a project	**Parking sensor**: zones with comparisons	The core of our rear alarm, see Test 06	**Must do**

Same code, different pins

The website uses its own pins. The car uses other pins because the L298N, the ESP32-CAM link and the sensor all need space. **The code is the same, only the numbers at the top change.**

Part	Website pin	Robot car pin	Why the car is different
HC-SR04 TRIG / ECHO	`D9` / `D10`	`D13` / `D12`	D9 and D10 drive the L298N (IN2, IN1)
Buzzer	`D8`, passive, `tone()`	`A3`, active, `digitalWrite`	`tone()` uses Timer2 and would break the PWM on D11 (ENA)
LED	`D13` (built-in)	`A0` red, `A2` white, `A4` headlights	D13 is the sensor's TRIG on the car
Motor PWM	`~D6` → TIP120	`~D11` ENA, `~D6` ENB → L298N	Two sides of the car, and the L298N can reverse
Servo	`D9`	`A1`	Optional scanning radar
Serial Monitor	9600 baud	115200 baud	Faster, so printing does not slow the car down

Active or passive buzzer?
An **active** buzzer beeps by itself when its pin is HIGH (this is the one in our parts list). A **passive** buzzer, like the website's, needs `tone()`. On the car `tone()` would stop the left motors working properly, so keep the active buzzer.

Bridge exercise: Test 06, parking alarm on the bench

Lesson 16 lights a traffic light. Its specifications even say *"Next step: add a buzzer from lesson 8"*. `tests/06_parking_alarm_bench` does exactly that, on the robot car's pins, so the wires stay the same when you move them to the car.

From	To
HC-SR04 VCC / GND	`5V` / `GND`
HC-SR04 TRIG	`D13`
HC-SR04 ECHO	`D12`
Red LED (+) via 220 Ω	`A0`
White LED (+) via 220 Ω	`A2`
Buzzer (+)	`A3`
LED (−), buzzer (−)	`GND`

1. Wire the parts on a breadboard with the UNO only (no battery, no motors).
2. Open `tests/06_parking_alarm_bench`, select **Arduino Uno**, upload.
3. Open the Serial Monitor at **115200**.
4. Move your hand slowly towards the sensor and watch the bar grow.

Distance	Zone	Buzzer + red LED	Serial Monitor
more than 100 cm	SAFE	Off	`150 cm SAFE ##`
100 → 20 cm	WARN	Beeps, faster as you get closer (900 → 120 ms)	`55 cm WARN ######`
20 cm or less	STOP!	Continuous	`12 cm STOP! ##########`

Worksheet

#	Question	Your answer
1	Lesson 2: what does `digitalWrite(pin, HIGH)` do to the pin's voltage?	
2	Lesson 5: `analogWrite(pin, 128)`. About what percentage of full power is that?	
3	Lesson 7: the echo lasted 1160 µs. How far is the obstacle? Show your calculation.	
4	Lesson 7: why divide by 58 and not by 29? (Hint: the sound travels there *and back*.)	
5	Lesson 8: what is the difference between an active and a passive buzzer?	
6	Lesson 10: why can't an Arduino pin power a motor directly? What does the L298N add?	
7	Lesson 16: write the comparison that is true only when the obstacle is between 20 and 100 cm.	
8	Test 06: with `map(cm, 20, 100, 120, 900)`, what is the beep period at 60 cm?	

Challenges (extra)

- Add the green / yellow / red traffic light from lesson 16 on free pins and keep the buzzer working.
- Change `WARN_CM` and `STOP_CM`. What happens if STOP is bigger than WARN? Fix it in code.
- Lesson 12: mount the sensor on a servo and print the distance for 5 angles: a text radar.

Checklist

Done	Item
☐	Lessons 1, 2, 5, 7, 8, 10 and 16 run on the website
☐	Worksheet answered
☐	Test 06 works on a real UNO: SAFE / WARN / STOP! zones correct
☐	I can explain why the car's pins differ from the website's

Student _____ Teacher _____ Date _____

Learn Arduino website designed by x.com/DilumSanjaya. This module only links to it; the lessons belong to their author.